
DyGRO-VLA: Cross-Task Scaling of Vision–Language–Action Models via Dynamic Grouped Residual Optimization

Sixu Lin^{1 2} Yunpeng Qing³ Litao Liu⁴ Ming Zhou⁵ Ruixing Jin¹ Xiaoyi Fan² Guiliang Liu^{1 6†}

Abstract

Recent progress in Reinforcement Learning (RL) provides a principled approach to optimizing Vision-Language-Action (VLA) models, facilitating a shift from trajectory imitation to active learning in the task environment. Despite improvements in control precision, most RL optimizers remain task-specific, which reduces VLA models from generalist controllers to policies that overfit to a narrow set of tasks. In this study, we conduct an in-depth analysis of this phenomenon and highlight the importance of cross-task feature representations for improving the generalizability of VLA models. Motivated by this finding, we introduce DyGRO-VLA, a two-stage optimization framework that 1) effectively captures cross-task latent representations based on information-theoretic principles, and 2) dynamically refines policy optimization via a mixture-of-RL-residuals. DyGRO-VLA enables the RL optimizer to exploit task-relevant latent information while strategically mitigating adverse interference on the learned representations throughout the optimization process. We evaluate our approach on LIBERO, RoboTwin2 benchmarks, and further validate it on real world, demonstrating consistent improvements over strong baselines under multi-task training and distribution shift. Our project page is available at [DyGRO-VLA](#).

1. Introduction

Under the rapid development of large-scale foundation models, Vision-Language-Action (VLA) models have emerged

as a promising paradigm for generalist robot manipulation policy by unifying perception, language understanding, and action control (Ma et al., 2024). Compared with lightweight task-specific policies (Chi et al., 2023; Zhao et al., 2023), VLAs typically offer stronger semantic grounding and better multi-task transfer, largely due to their higher representational capacity and the ability to reuse shared structure across heterogeneous tasks. As a result, VLAs are widely viewed as an important step toward general-purpose robots that can solve diverse real-world tasks with a unified model (Black et al., 2024; Intelligence et al., 2025).

Despite these advances, training a high-performing generalist policy solely from offline demonstrations remains challenging. Compared to datasets for language modeling and visual understanding, robotic datasets are relatively small, less diverse, and costly to collect (Xing et al., 2025), especially for long-tail scenarios that are less likely to be encountered in practice. To mitigate the data scarcity issues, recent studies incorporate online Reinforcement Learning (RL) to optimize VLA in the post-training (Chen et al., 2025b; Lu et al., 2025; Li et al., 2025a; Zang et al., 2025).

While RL-based optimization can effectively improve success rates on individual dexterous manipulation tasks (Luo et al., 2024; 2025), these gains often come at the expense of losing cross-task scalability. Our in-depth study reveals that as the number of tasks increases, the performance of RL optimization degrades substantially (see Figure 2). In some cases, it even harms the original VLA model that was pre-trained on offline data (see Figure 1). We find that this failure mode arises because the RL optimizer can distort the shared representation learned during pretraining, effectively isolating task-specific experience and reducing cross-task knowledge sharing (see Figure 3). This distortion leads to catastrophic forgetting and weakens cross-task adaptation. Therefore, balancing the trade-off between dexterity and generalizability remains a significant open challenge.

To address this challenge, we propose Dynamic Grouped Residual Optimization (DyGRO), a cross-task optimizer for VLA models. Specifically, DyGRO-VLA follows a two-stage optimization paradigm with the following objectives:

1) *Learning shared representations across tasks.* DyGRO-

[†]Corresponding author. ¹School of Data Science, The Chinese University of Hong Kong (Shenzhen) ²Jiangxing Intelligence Technology Inc. ³Zhejiang University ⁴Rutgers University-New Brunswick ⁵Shanghai AI Laboratory ⁶Shenzhen Loop Area Institute. Correspondence to: Guiliang Liu <liuguiliang@cuhk.edu.cn>.

VLA follows the Information Bottleneck (IB) principle (Tishby et al., 1999) to extract action-relevant features from visual observations and robot states. By preserving useful information for guiding robot motion while discarding nuisance factors (e.g., background clutter and lighting variations), these features can be more effectively shared across tasks. Practically, to make IB learning computationally tractable, we derive a variational lower bound tailored to offline robot behavior modeling and use it to jointly train the feature representation and action prediction models.

2) *Finetuning with Mixture-of-RL-Residuals (MoRR)*. To optimize the policy without disrupting the shared cross-task representation, we design the RL policy to predict a residual as refinements of the prediction from the action model (Johannink et al., 2019). However, classic residual learning is typically tailored to a single task. To generalize across a variety of tasks, we strategically combine residual refinements from a mixture of RL policies, where the mixture weights are determined by a dynamic routing mechanism. Consequently, while each policy only specializes in a subset of tasks, the router can adaptively select and compose them to produce the most effective refinement for the current tasks. To implement MoRR, we introduce a task-embedding method that provides task information to the router, based on which we design an online training scheme that jointly updates the RL residual policies and the routing model.

To validate the effectiveness of DyGRO-VLA, we conduct extensive experiments across diverse multi-task manipulation benchmarks, including LIBERO (Liu et al., 2023) and RoboTwin2 (Chen et al., 2025a). Across all settings, DyGRO-VLA consistently outperforms state-of-the-art baselines, achieving the highest average success rate of 97.1% on LIBERO, an absolute gain of +4.4% over its offline base model—and demonstrating substantial improvements on the most challenging LIBERO-Long suite (+9.8%). Furthermore, in real-world evaluations on RoboTwin2, DyGRO-VLA attains the best overall success in simulation (79.2%) and surpasses RFT baselines under Sim2Real transfer, particularly on complex bimanual and long-horizon tasks. These results highlight DyGRO-VLA as a scalable and robust solution for efficient multi-task reinforcement fine-tuning and real-world robotic manipulation.

2. Related Work

Multi-task Learning. A central challenge in multi-task learning is the interference among task gradients, which can hinder optimization efficiency. Prior work has sought to mitigate this issue through sample- or task-level reweighting strategies (Peng et al., 2023). Broadly, existing approaches fall into two categories: (1) designing reweighting schemes to balance the relative contributions of different tasks (Sener & Koltun, 2018; Parisotto et al., 2015; Yu et al., 2020), and

(2) adjusting per-task gradient directions to reduce conflicts and encourage cooperative learning (Yu et al., 2020; Liu et al., 2021; Chen et al., 2020; Wang et al., 2020; Bohn et al., 2024). More recently, token-level MoE routing methods have also been proposed to address gradient conflicts at finer granularity (Yang et al., 2024). (Yang et al., 2020; Huang et al., 2024; Kong et al., 2025; Wu et al., 2025) utilize routing strategy to resolve multi-task gradient conflicts.

Reinforcement Learning for Vision-Language-Action Models. Recent studies demonstrate that reinforcement finetuning can significantly improve VLA performance in manipulation tasks (Huang et al., 2025; Liu et al., 2025; Chen et al., 2025b; Guo et al., 2025; Tan et al., 2025; Li et al., 2025a; Zang et al., 2025). (Liu et al., 2025) analyzes the generalization benefits brought by RL to VLAs, while (Chen et al., 2025b) successfully deploys reinforced fine-tuning in real-world sparse-reward settings, showcasing effective adaptation in challenging robotic scenarios. In addition, (Guo et al., 2025) proposes an iterative learning paradigm that stabilizes training and improves efficiency. Different from these works, our method emphasizes the *scalability* of VLA post-training. While prior approaches focus on task-level performance gains or real-world adaptation, we argue that RL should preserve the intrinsic generalization ability of large VLAs, fundamentally distinguishing them from lightweight, task-specific models.

3. Problem Formulation

In this section, we discuss the problem of cross-task finetuning of VLA policies using RL methods and formulate the key challenges with empirical evidence.

Multi-task Reinforcement Learning. We consider solving a distribution of tasks $\zeta \sim p(\zeta)$ with a single policy π . In this setting, each task induces a POMDP $\mathcal{M}^\zeta = (\mathcal{S}^\zeta, \mathcal{A}, P^\zeta, R^\zeta, \mathcal{O}^\zeta, \rho^\zeta, \gamma)$. A task begins at an initial state $s_0 \sim \rho^\zeta$. At each timestep t , the agent receives an observation $o_t \in \mathcal{O}^\zeta$ composed of RGB images, robot proprioceptive information, and a natural language instruction. The agent then selects an action $a_t \in \mathcal{A}$ from the shared primitive action space, receives a reward $r_t \sim R^\zeta$, and transitions to the next state s_{t+1} following the transition function P^ζ . For brevity, we assume sparse rewards for robotic control: the agent receives a success reward of +1 upon task completion. The objective of the policy π is to maximize the expected discounted return across the task distribution:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\zeta \sim p(\zeta)} \mathbb{E}_{\tau \sim \pi, \mathcal{M}^\tau} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]. \quad (1)$$

Striving for temporal abstraction, we implement $\pi(a_t \mid o_t)$ as *action chunking* policy (Zhao et al., 2023; Huang et al., 2025; Li et al., 2025b). An action chunk is defined as $a_t = a_{t:t+h-1} \in \mathcal{A}^h$.

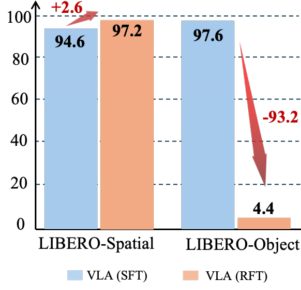


Figure 1. Catastrophic Forgetting. RFT may improve the trained tasks but leads to increasing performance drops on other tasks.

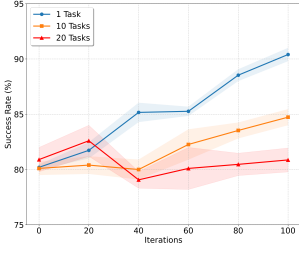


Figure 2. Multi-Task Conflict. As the number of tasks increases, RFT becomes unstable and increasingly ineffective.

Catastrophic Forgetting in Cross-Tasks Adaptation. Developing scalable and general-purpose VLA models is a central objective in embodied robotic control (Black et al., 2024; Shi et al., 2025; Intelligence et al., 2025). Although recent advances in reinforcement learning (RL) optimization have substantially improved the dexterity and task success rates of VLA policies (Li et al., 2025a; Lu et al., 2025), these models often remain highly task-specific. In particular, such RL optimizers tend to overfit to individual tasks and consequently struggle to retain prior knowledge in continual multi-task learning settings.

For empirical evidence, we test on LIBERO (Liu et al., 2023) benchmark. We conduct empirical analysis using the VLA model introduced in Section 4.1. As shown in Figure 1, when training a VLA on LIBERO-Spatial, we observe that performance on unrelated tasks in LIBERO-Object rapidly declines as training progresses. The success rate drops significantly, indicating that specialization on spatial tasks erodes the model’s capacity to handle object-centric tasks.

Delving into the phenomenon of catastrophic forgetting, we observe that a fundamental prerequisite for the scalability of VLA models is the development of a shared feature space capable of encoding useful information across diverse tasks. However, during fine-tuning on a specific task, the RL agent often distorts this shared representation by isolating its representation of its experience from others. As shown in Figure 3, single-task RFT drives the tuned task’s representations to form an isolated cluster, indicating a drift away from the shared feature space and reduced alignment with other similar tasks. It eventually results in a deterioration of VLA’s cross-task competencies.

Rethinking RL Post-training for VLA. Revisiting the objective of VLA models, the ultimate goal is to learn a generalist control policy capable of performing effectively across diverse tasks. In practice, the variations among these tasks, including differences in time horizons, the number and types of target objects, and the visual, physical, and dynamic properties of the environments, are substantial (Kroemer et al., 2021; Kawaharazuka et al., 2025). Given the

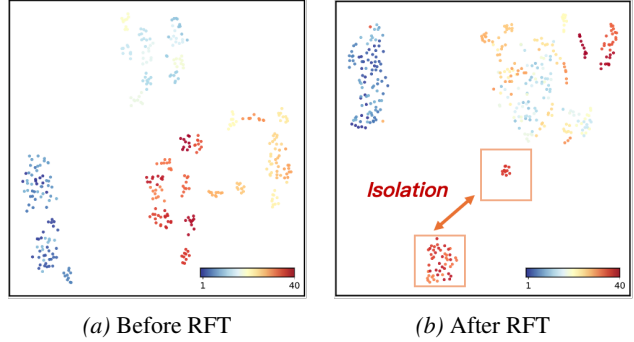


Figure 3. t-SNE visualization of the shared feature space on LIBERO. We embed representations from 40 tasks (10 samples per task) before and after single-task RFT.

inherently task-driven nature of RL optimizers, they are prone to overfitting to specific tasks and forgetting previously learned ones, as discussed above. This tendency naturally introduces a trade-off between achieving high task success rates and maintaining cross-task scalability. This trade-off poses a significant challenge for the direct application of RL optimizers in VLA models. To study this effect, we perform RL post-training with a classic actor–critic algorithm, Soft Actor-Critic (SAC) (Haarnoja et al., 2018). We evaluate three multi-task regimes (3 random seeds): (i) a single task from the LIBERO Spatial suite, (ii) 10 tasks from LIBERO Spatial, and (iii) 20 tasks from LIBERO Spatial and LIBERO Goal. As illustrated in Figure 2, the average success rate becomes increasingly unstable as the number of tasks grows during RL post-training. In particular, when trained on 10 tasks, only marginal improvements are observed, whereas with 20 tasks, the average success rate declines sharply, underscoring the limitations of current multi-task RL optimizers.

In this study, unlike prior works (Chen et al., 2025b; Lu et al., 2025; Li et al., 2025a) that focus on how RL optimizes the performance of individual tasks (i.e., for each task ζ , $\max \mathbb{E}_{\mathcal{M}_\zeta} [\sum_{t=0}^{\infty} \gamma^t r_t]$), we examine the collective success rates of tasks as a group (i.e., for all $\zeta \sim p(\zeta)$, $\max_{|\zeta|} \frac{1}{|\zeta|} \sum_{\zeta} \mathbb{E}_{\mathcal{M}_\zeta} [\sum_{t=0}^{\infty} \gamma^t r_t]$) and introduce a novel RL framework that effectively addresses the trade-off between accuracy and scalability in VLA post-training.

4. DyGRO-VLA

To achieve scalable cross-task performance, we propose DyGRO-VLA, a RL optimization framework for VLA models. Similar to recent VLA post-training methods (Lu et al., 2025; Li et al., 2025a;b), DyGRO-VLA follows an *offline pretraining-to-online fine-tuning* paradigm. However, unlike prior works targeting individual task performance, DyGRO-VLA seeks to generalize across multiple tasks by 1) learning shared representations in pretraining and 2) optimizing the performance all tasks concurrently during post-training.

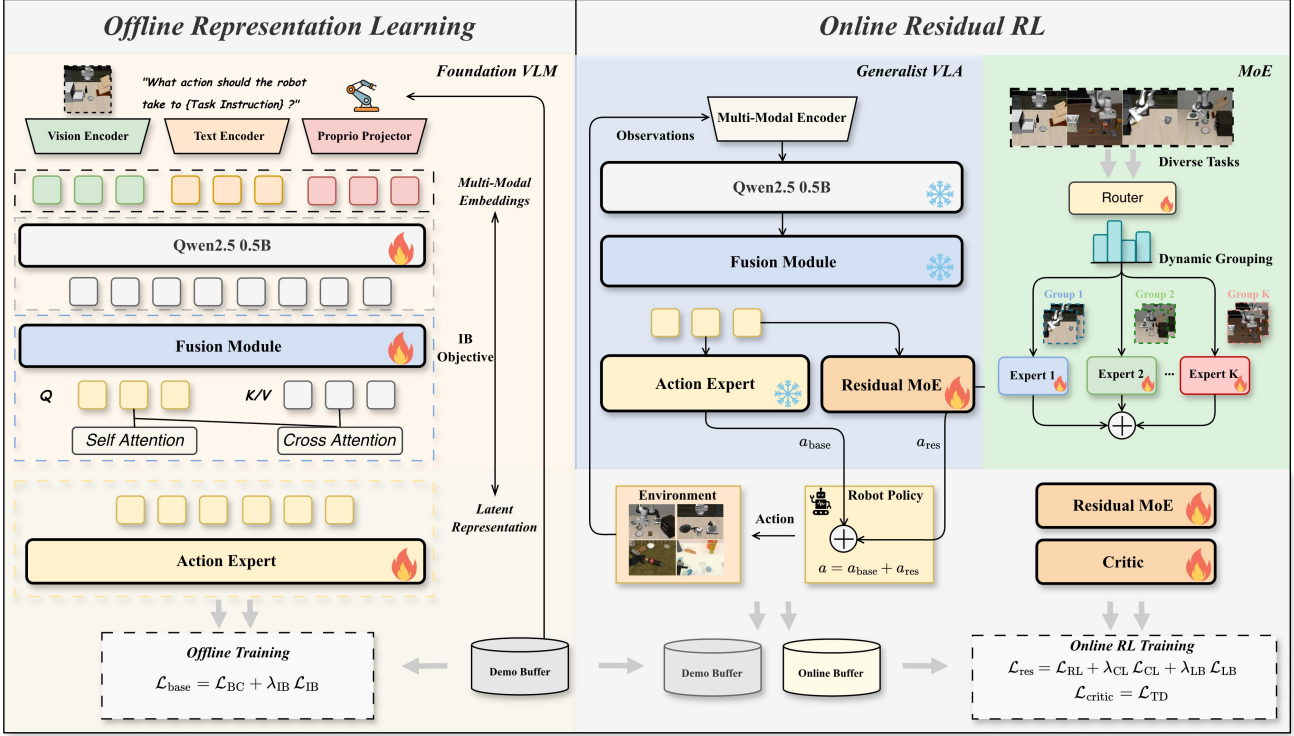


Figure 4. **Method pipeline.** DyGRO-VLA follows a two-stage training recipe. **1) Offline stage:** we train the VLA backbone to predict actions while learning a compact latent representation via an information-bottleneck (IB) objective. **2) Online stage:** we freeze the VLA backbone and optimize the residual MoE in online multi-task settings, serving as a residual compensation module on top of the base model to further improve overall multi-task ability and generalization.

4.1. Offline Pre-Training for Cross-Task Representation

The goal of pretraining is to learn a cross-task latent representation $p(z | o)$ that not only captures essential information from the observation o but also facilitates the learning of the action model $\pi(a | z)$ across diverse tasks.

Latent Feature Extraction. DyGRO-VLA implements $p(z | o)$ using a VLM backbone coupled with a fusion module to encode exteroception from sensors and commands, and to fuse these features with proprioceptive information.

At each time step t , the backbone processes images from both wrist and third-view cameras $\mathcal{X}_t$ together with an instruction $\mathcal{L}_t$. Visual observations are encoded using DI-NOv2 (Oquab et al., 2024) and SigLIP (Zhai et al., 2023), and subsequently fused with tokenized language representations through a Qwen2.5-0.5B LLM (Team, 2024), yielding a sequence of hidden states. These hidden states are then partitioned into *conditioning tokens* h_t^{cond} and *action tokens* h_t^{act} , where h_t^{cond} integrate vision and language.

Upon completing the encoding stage, the fusion module initializes a set of learnable action-query tokens q_t^a and fuses them with h_t^{cond} , h_t^{act} , and the proprioceptive input p_t to produce h_t^{fuse} (Wang et al., 2025). Appendix A.1 covers the details.

Learning Task-Sharing Representation. To effectively extract task-sharing features $\tilde{z}_T$ from the fused latent features h_t^{fuse} , DyGRO-VLA designs the learning objective based on the principle of Information Bottleneck (IB) (Tishby et al., 1999) and designs the learning objective as follows:

$$\max_{p_\theta(z|o)} I(Z; A) - \lambda_{\text{IB}} I(Z; O), \quad (2)$$

where the probabilistic model $p_\theta(z|o)$ first extracts h_t^{fuse} from o and then transforms it into $\tilde{z}_T$. In essence, IB principle formalizes representation learning as a trade-off between *compression* and *task relevance*. Given an input observation O and a latent representation Z , the IB framework seeks a stochastic encoder that retains information useful for downstream robot action prediction, such as spatial and morphological features of the target object, while discarding nuisance factors such as background and lighting conditions. This is achieved by maximizing $I(Z; A)$ (I denotes the mutual information between two variables) while minimizing $I(O; Z)$, weighted by λ_{IB} .

However, quantifying mutual information directly is intractable for high-dimensional inputs. Therefore, we employ a variational approximation to derive a lower bound of Objective 2 and propose the following objective:

Proposition 4.1 (Variational Information Bottleneck for Ac-

tion Regression). Let Z be a latent representation produced by an encoder $p_\theta(z|o)$, and let $\pi_\theta(a|z)$ denote a probabilistic action decoder. Then, maximizing the IB objective (2) is equivalent to minimizing the following loss term:

$$\mathcal{L}_{base} = \mathbb{E}_{p_\theta(z|o)}[-\log \pi_\theta(a|z)] + \lambda_{IB} [\mathbb{E}_{P_{OZ}}[T_\psi(o, z)] - \log \mathbb{E}_{P_{OP_Z}}[e^{T_\psi(o, z)}]] \quad (3)$$

where 1) $T_\psi : \mathcal{O} \times \mathcal{Z} \rightarrow \mathbb{R}$ denotes a neural critic, and 2) P_{OZ} and P_{OP_Z} denote the joint distribution of O and Z , as well as the product of their marginals.

Appendix B shows the proof. In practice, we estimate the $\mathbb{E}_{P_{OZ}}[T_\psi(o, z)]$ using paired samples in each mini-batch, and approximate sampling from the product of marginals P_{OP_Z} by randomly permuting z within the same mini-batch (pairing o_i with $z_{\pi(i)}$ for a permutation π) (Belghazi et al., 2018). For stability, we reuse the same permutation within each optimization step.

Finally, to enable accurate action prediction, we parameterize the policy $\pi_\theta(a|z)$ using the action head architecture from prior work (Kim et al., 2025). Given the fused latent representation $z \in \mathcal{Z}$, the action head maps it to continuous control commands and outputs an action chunk of horizon h for execution. Formally, the base policy is a mapping $\pi_\theta^{base} : \mathcal{Z} \rightarrow \mathcal{A}^h$.

Accordingly, the maximum likelihood term in objective (3) can be approximated by regressing vector fields of fixed conditional probability path.

By combining $\pi_\theta^{base}(a|z)$ and $p_\theta(z|o)$, we obtain a shared base policy trained on offline demonstrations to acquire general multi-task competence.

4.2. Online Finetuning via Mixture of RL Residuals

Upon learning the task-sharing representation $p_\theta(z|o)$, we proceed to optimize the action prediction model $\pi_\theta^{base}(a|z)$ using an RL optimizer. However, as discussed in Section 3, although prior RL fine-tuning methods enhance the VLA’s dexterity on individual tasks, these improvements come at the cost of distorting the learned task-sharing representation. Such distortion compromises the VLA model’s ability to scale effectively across multiple tasks.

To address the trade-off between control dexterity and cross-task scalability, DyGRO-VLA employs a dynamic Mixture-of-RL-Residuals (MoRR) for multi-task RL optimization.

Learning Task Embeddings. DyGRO-VLA learns multiple RL policies and leverages their combination to optimize task performance. A key prerequisite for this process is determining which policies should be involved to handle a given task. To this end, we develop a task embedding denoted as $\tilde{z}_T$. Notably, unlike the latent representation

$p_\theta(z|o)$, which primarily captures decision-relevant patterns, the task embedding $\tilde{z}_T$ is designed to encode task-identification patterns. During training, task IDs are used only as supervision to shape the embedding space, while the router is conditioned on $\tilde{z}_T$ rather than directly consuming explicit environment labels (e.g., task IDs, names, or object categories). This design encourages the task embedding to capture semantic task features and allows DyGRO-VLA to select appropriate experts from the learned routing space at inference time.

Inspired by contrastive learning (Oord et al., 2018; Wu et al., 2025), we design the following loss function:

$$\mathcal{L}_{CL}(\psi) = -\mathbb{E}_{(\tilde{z}_T, \zeta)} \left[\log \frac{\exp(\text{sim}(\tilde{z}_T, e_\zeta)/\tau)}{\sum_{\zeta'=1}^N \exp(\text{sim}(\tilde{z}_T, e_{\zeta'})/\tau)} \right], \quad (4)$$

where N is the number of training tasks, $e_\zeta = \text{Emb}(\zeta)$ denotes the learnable task prototype associated with task ID ζ , τ is a temperature hyperparameter, and $\text{sim}(\mathbf{a}, \mathbf{b}) = \mathbf{a}^\top \mathbf{b}$ is the similarity function. In practice, the MoE router is conditioned on the task embedding $\tilde{z}_T$, while the task prototypes e_ζ are only used during contrastive training to shape the embedding space and are not directly involved in routing decisions.

Mixture of Residuals with Dynamic Routing. Based on the task embeddings, DyGRO-VLA utilizes a Mixture-of-Experts (MoE) module (Jacobs et al., 1991; Shazeer et al., 2017) to refine the action chunk across multiple tasks.

Specifically, given a task embedding $\tilde{z}_T$, a router $h(\cdot)$ produces gating logits over N residual experts $\{\pi_i^\Delta\}_{i=1}^N$ and activates only the top- m of them. The corresponding gating weights are then used to combine the outputs of the selected experts as follows:

$$\Delta \mathbf{a} = \sum_{i=1}^m \omega_i(\tilde{z}_T) \tilde{\mathbf{a}}_i \quad \text{s.t.} \quad \tilde{\mathbf{a}}_i \sim \pi_{i,\psi}^\Delta(\tilde{\mathbf{a}}_i | z, \mathbf{a}_{base}) \quad (5)$$

where 1) $\{\omega_i(\tilde{z}_T)\}_{i=1}^m$ are the top m gating weights predicted by the router and 2) $\tilde{\mathbf{a}}_i$ is the residual chunk predicted by the i -th expert, conditioned on the latent features z and the base action chunk $\mathbf{a}_{base}$ (predicted by the pre-trained policy π_θ^{base}). This residual parameterization preserves the base policy as a strong prior while enabling specialized experts to perform targeted corrections.

Learning the Residual RL Policies. Unlike prior residual-policy methods that attach an external residual controller to a frozen base policy (Xiao et al., 2025b; Yuan et al., 2024), we integrate a residual MoE into the base architecture by conditioning each expert on the same VLM latent representation used by the base action head. This avoids introducing a separate perception encoder for the residual branch, improving compute efficiency and reducing redundant re-encoding.

The RL loss of residual policy is formulated as:

$$\mathcal{L}_{\text{RL}}(\psi) = \mathbb{E} \left[\log \pi_{\psi, \theta}(\mathbf{a}_t | s_t) - \frac{1}{K} \sum_{k=1}^K Q_{\theta_k}(s_t, \mathbf{a}_t) \right]. \quad (6)$$

where 1) $\pi_{\psi, \theta}$ denotes the augmented policy that merges the predicted action refinement $\Delta \mathbf{a}$ (Eq. (5)) with action samples from the base policy $\pi_{\theta}^{\text{base}}(a | z)$ such that $\forall \mathbf{a} \sim \pi_{\psi, \theta}, \mathbf{a} = \Delta \mathbf{a} + \mathbf{a}_{\text{base}}$; and 2) $Q_{\theta_k}(s_t, \mathbf{a}_t)$ denotes the k -th critic in a K -ensemble of action-value functions, parameterized by θ_k . The critic objective is detailed in Appendix A.2.

To mitigate routing instability and prevent expert collapse in the MoE policy, we regularize the gating distribution to encourage balanced expert utilization. Let $\hat{\omega}_n \in \Delta^{N-1}$ denote the predicted router distribution for the n^{th} task embedding $\tilde{z}_{T,n}$ during training, and let $\hat{\omega}_{\text{avg}} = \frac{1}{B} \sum_{n=1}^B \hat{\omega}_n$ be the mini-batch average (B is the minibatch size). We introduce an entropy-based load-balancing regularizer as:

$$\mathcal{L}_{\text{LB}}(\psi) = \sum_{i=1}^N \hat{\omega}_{i, \text{avg}} \log(\hat{\omega}_{i, \text{avg}} + \epsilon), \quad (7)$$

where ϵ is a small constant for numerical stability. The *sparse* mixture weights $\omega_i(\cdot)$ used in Eq. (5) are obtained by selecting top- m $\hat{\omega}_i$ followed by renormalization. Minimizing $\mathcal{L}_{\text{LB}}$ is equivalent to maximizing the entropy of the routing probabilities, thereby preventing collapse to a single residual policy. We add $\mathcal{L}_{\text{LB}}$ to the policy objective. Therefore, the overall training objective of MoRR policy is:

$$\mathcal{L}_{\pi}(\psi) = \mathcal{L}_{\text{RL}}(\psi) + \lambda_{\text{CL}} \mathcal{L}_{\text{CL}}(\psi) + \lambda_{\text{LB}} \mathcal{L}_{\text{LB}}(\psi), \quad (8)$$

where λ_{CL} and λ_{LB} control the weights of the corresponding loss terms.

Difficulty-Aware Sampling. Inspired by (Lu et al., 2025; Florensa et al., 2017; 2018), we bias task sampling toward under-solved tasks. Specifically, given the empirical success rate sr_j of task ζ_j , we define $P(\zeta_j) \propto \exp(\max(0, \text{target} - \text{sr}_j)/\tau)$, which increases the sampling probability only when $\text{sr}_j < \text{target}$ and saturates otherwise. This asymmetric design focuses training on partially solvable yet still challenging tasks while retaining minimal exposure to mastered tasks.

Practical Implementation. In practice, we maintain two replay buffers: an offline buffer $\mathcal{D}_{\text{offline}}$ and an online buffer $\mathcal{D}_{\text{online}}$. Following RLPD-style sampling (Ball et al., 2023), we uniformly mix offline and online transitions for each gradient update. To ensure stable initialization of online residual RL, we warm up both the critic and policy prior to online interaction, as in Cal-QL (Nakamoto et al., 2023). We further use BC regularization (Fujimoto & Gu, 2021) and an advantage-filtered actor update (Nair et al., 2020) to stabilize residual policy learning. Pseudocode are provided in Appendix D.

5. Experiments

5.1. Experiments Settings.

Benchmark. 1) LIBERO (Liu et al., 2023) is a lifelong learning benchmark with five suites spanning 130 tasks. *LIBERO-Spatial* evaluates spatial generalization by varying object placements; *LIBERO-Object* tests object generalization by placing different objects into a box; *LIBERO-Goal* measures diverse operations in a fixed environment; and *LIBERO-Long* contains ten long-horizon tasks across varied scenes. 2) RoboTwin2.0 (Chen et al., 2025a) is a dual-arm manipulation benchmark designed for cross-embodiment evaluation. We select four representative tasks to assess real-world transfer performance and practical deployability.

Baseline. To assess each model’s multi-task capability, we train all baseline methods on a unified training set formed by mixing data from all four suites with full shot data. To evaluate scalability and efficiency across model scales, we consider both generalist large VLA models and light-weight multi-task policies. Specifically, we include five recent large-scale VLA baselines reported in Table 1, including Octo (Team et al., 2024), OpenVLA (Kim et al., 2024), SpatialVLA (Qu et al., 2025), π_0 -FAST* (Pertsch et al., 2025), and π_0 (Black et al., 2024). These models are fine-tuned from their released pretrained checkpoint when applicable. In addition, we evaluate two representative light-weight baselines, Diffusion Policy (Chi et al., 2023) and MT-ACT (Bharadhwaj et al., 2023). All baselines are trained under the same mixed-suite protocol for a fair comparison. To provide an initial sanity check for our approach, we also train the base DyGRO-VLA policy using supervised fine-tuning (SFT) under the same setting. Following common practice, we adopt LoRA (Hu et al., 2022) with rank 64 for this SFT baseline.

5.2. Main Results

Results Analysis. Table 1 reports success rates under the 4-suite co-training setting, evaluated on each LIBERO suite. Overall, DyGRO-VLA achieves leading or comparable performance across all suites. Compared with the offline SFT base model, our reinforcement fine-tuning consistently improves Spatial, Object, and Goal performance, and yields a clear average gain (92.7% $\rightarrow$ 97.1%). Notably, the largest improvement appears on the most challenging suite, *LIBERO-Long*, where success increases from 85.2% to 95.0%, indicating substantially enhanced long-horizon robustness. DyGRO-VLA remains competitive with prior multi-task VLA models under the same multi-task protocol.

Reinforcement Fine-Tuning (RFT) Baseline Comparisons. We evaluate DyGRO-VLA against representative RFT baselines on *LIBERO-Long* (Table 2) to characterize post-training performance. Since most prior RFT methods

Table 1. **LIBERO**. Success rate (%) on LIBERO Benchmark. **Best** results are in **bold** and **second-best** results are underlined.

Method	Spatial	Object	Goal	Long	Avg.
Multi-task SFT Models					
Diffusion Policy (From Scratch) (Chi et al., 2023)	59.6	73.8	51.6	41.0	56.5
MT-ACT (From Scratch) (Bharadhwaj et al., 2023)	50.2	72.0	60.2	50.2	58.2
Octo (Team et al., 2024)	78.7	85.5	84.2	51.0	74.9
OpenVLA (Kim et al., 2024)	82.1	87.3	77.4	51.8	74.7
SpatialVLA (Qu et al., 2025)	88.2	89.9	78.6	55.5	78.1
π_0 -FAST* (Pertsch et al., 2025)	96.4	<u>96.8</u>	88.6	60.2	85.5
π_0 (Black et al., 2024)	98.0	<u>96.8</u>	<u>94.4</u>	<u>88.4</u>	<u>94.4</u>
DyGRO-VLA (SFT)	95.4	96.0	93.8	85.0	92.6
Multi-task RFT Models					
DyGRO-VLA (Offline)	95.6	96.0	94.0	85.2	92.7
DyGRO-VLA	<u>97.6</u>	98.6	97.2	95.0	97.1
Δ	+2.0	+2.6	+3.2	+9.8	+4.4

 Table 2. **RFT Comparisons**. Success rate (%) on LIBERO-Long.

Method	LIBERO-Long
TGRPO (Chen et al., 2025c)	59.2
GRAPE (Zhang et al., 2024)	57.2
VLA-RL (Lu et al., 2025)	59.8
World-Env (Xiao et al., 2025a)	57.8
RIPT-VLA (Tan et al., 2025)	93.8
SimpleVLA-RL (Li et al., 2025a)	91.7
RLinf (Zang et al., 2025)	94.0
DyGRO-VLA	95.2

are tailored to single-task or single-suite training and are not readily applicable to joint co-training across all four LIBERO suites, a direct four-suite comparison would be confounded by mismatched training assumptions. We therefore adopt a suite-wise evaluation protocol and report results consistently across suites. The results show DyGRO-VLA achieves the best performance with a success rate of 95.2%.

Conflict Mitigation via Dynamic Grouping. We analyze gradient conflicts using four representative tasks with contrasting interaction semantics, consisting of two *Open* tasks and two *Close* tasks (Open I/II, Close I/II). We quantify inter-task interference via pairwise cosine similarity of per-task gradients (Yu et al., 2020), comparing a single MLP residual head against our MoRR. As shown in Figure 5, gradients are positively aligned within the *Open* and *Close* pairs, but exhibit negative correlations across *Open* $\leftrightarrow$ *Close*, indicating strong interference under a shared MLP head. MoRR reduces these negative similarities while maintaining within-group alignment, suggesting that MoE routing with dynamic grouping effectively mitigates cross-task conflicts.

5.3. Ablation Study

Expert Number Ablation. To verify whether scaling the MoRR improves multi-task performance, we vary the number of experts and report results in Table 4. Increasing

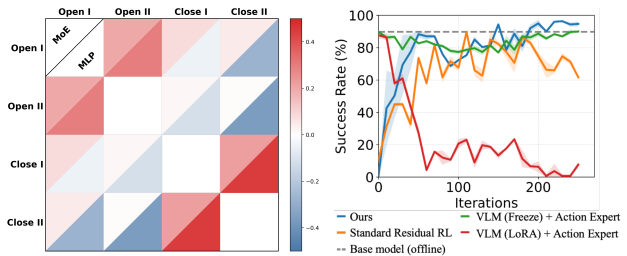


Figure 5. **Gradient Conflicts.** Figure 6. **Online Training Strategy Ablation.** Success rate versus online fine-tuning iterations for different strategies. Curves are averaged over 3 seeds; the dashed line denotes the offline base model.

experts consistently boosts success: a single expert already reaches 93.4 average success, 4 experts improve to 97.0 with gains across all suites, and 8 experts achieve the best overall average performance (97.1 avg.), topping *Spatial* and *Goal* while tying the best *Long* result. Notably, the largest gain is on long-horizon tasks, suggesting that additional experts enable better specialization and mitigate cross-task interference.

Component Ablation. To assess the impact of each component in DyGRO-VLA, we perform a component-wise ablation (Tab. 5) by removing one term at a time while keeping all other settings fixed. Removing IB leads to a consistent drop in performance (97.1 $\rightarrow$ 96.8), indicating that IB provides a reliable regularization benefit. Additionally, removing the InfoNCE-based contrastive learning objective leads to a pronounced degradation across suites, most notably on *Long* (95.0 $\rightarrow$ 90.4), reducing the overall average to 94.3, which underscores the role of discriminative task embeddings for stable routing and multi-task generalization. Disabling difficulty-aware sampling also reduces performance (97.1 $\rightarrow$ 96.4). Overall, the full model performs best, and these components contribute complementary gains.

Table 3. **RoboTwin2**. Success rate (%) on RoboTwin2 Benchmark. **Best** results are in **bold** and **second-best** results are underlined.

Method	Beat Block Hammer		Pick Dual Bottles		Stack Bowls Two		Place Empty Cup		Avg.	
	Sim	Real	Sim	Real	Sim	Real	Sim	Real	Sim	Real
OpenVLA-oft (SFT) (Kim et al., 2025)	54.0	15.0	32.0	25.0	88.0	70.0	50.0	<u>60.0</u>	56.0	42.5
OpenVLA-oft (RFT) (Li et al., 2025a)	<u>71.9</u>	30.0	<u>54.0</u>	40.0	92.0	90.0	<u>96.1</u>	<u>60.0</u>	<u>78.5</u>	<u>55.0</u>
DyGRO-VLA	72.2	30.0	57.0	40.0	<u>90.4</u>	90.0	97.0	70.0	79.2	57.5

Table 4. **Expert Number Ablation**. Success rate (%) on LIBERO.

Method	Spatial	Object	Goal	Long	Avg.
DyGRO-VLA (1)	95.2	96.4	93.6	88.2	93.4
DyGRO-VLA (4)	97.4	98.8	96.8	95.0	97.0
DyGRO-VLA (8)	97.6	98.6	97.2	95.0	97.1

Table 5. **Online Training Ablation**. Success rate (%) on LIBERO.

Method	Spatial	Object	Goal	Long	Avg.
DyGRO-VLA	97.6	98.6	97.2	95.0	97.1
w/o IB Objective	97.4	98.2	97.0	94.6	96.8
w/o CL Objective	95.2	97.2	94.2	90.4	94.3
w/o DA Sampling	97.4	98.4	97.0	93.0	96.4

Online Training Strategy Ablation. To examine the effect of online fine-tuning strategies, we conduct an ablation in which all methods are initialized from the same offline base model and evaluated over 3 seeds (Fig. 6). DyGRO-VLA achieves the highest success rate and the most stable improvement. In comparison, *Standard Residual RL* exhibits large oscillations and ends with noticeably lower performance. Freezing the VLM in *VLM (Freeze) + Action Expert* yields stable but modest gains, indicating limited adaptation capacity. Finally, *VLM (LoRA) + Action Expert* exhibits severe performance collapse, indicating that directly updating the backbone under large-scale multi-task online RL can substantially destabilize optimization.

5.4. Real-world Experiments

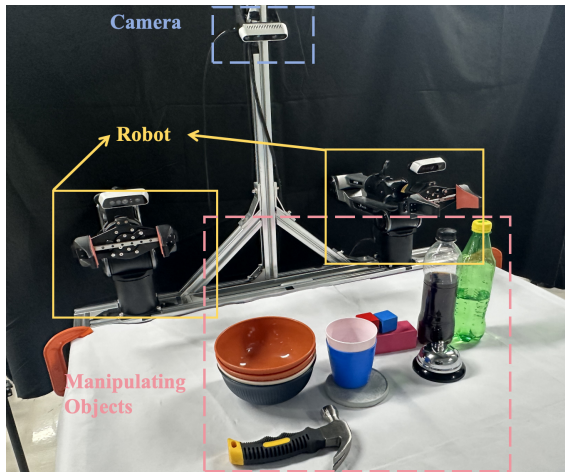


Figure 7. Real-world Platform for robotic manipulation.

Real-World Settings. We deploy our VLA model for real-world validation using a single Intel RealSense camera mounted in a head (top-down) view. DyGRO-VLA is trained in simulation and transferred to the real robot via a Sim2Real pipeline. Specifically, we follow the Sim2Real protocol of SimpleVLA-RL (Li et al., 2025a), applying domain randomization in simulation and transferring the resulting policy to the real world. We evaluate on four representative RoboTwin tasks: *Beat Block Hammer*, *Pick Dual Bottles*, *Stack Bowls Two*, and *Place Empty Cup*. We compare against OpenVLA-OFT (SFT) (Kim et al., 2025) and an RFT variant of OpenVLA-OFT trained with SimpleVLA-RL. All real-world evaluations use 20 trials per task, while simulation results follow the RoboTwin2 benchmark evaluation protocol.

Results Analysis. Our experiment results in Table 3 show that DyGRO-VLA achieves the best overall performance in simulation (79.2%) and matches or surpasses RFT baselines under Sim2Real transfer. In particular, DyGRO-VLA improves real-world success on *Pick Dual Bottles* and *Place Empty Cup*, while remaining competitive on *Stack Bowls Two*. These results validate the practical applicability of our method for deploying simulation-trained VLA policies in real-world settings.

5.5. Limitation

DyGRO-VLA has three main limitations: 1) *Online training sensitivity*: The online stage requires environment interaction and is sensitive to reward sparsity and hyperparameter tuning, which limits stability and sample efficiency. 2) *Task dependency in routing*: The routing mechanism relies on task embeddings trained with known task identities, reducing effectiveness in settings with ambiguous or unlabeled task boundaries. 3) *Limited real-world evaluation*: Current validation covers only a few tasks and a specific Sim2Real setup; broader testing across diverse tasks, embodiments, and sensing conditions is needed to assess generalization.

6. Conclusion

We investigate the scalability challenge of RL post-training for vision–language–action models, where multi-task online optimization can induce cross-task interference and catastrophic forgetting. To address this, we propose DyGRO-VLA, a two-stage framework that learns task-sharing representations from offline demonstrations and refines be-

havior online via dynamically routed residual RL experts. Across LIBERO, RoboTwin2, and real-world Sim2Real experiments, DyGRO-VLA improves multi-task performance and robustness over strong baselines, with notable gains on challenging tasks. These results highlight the value of preserving shared representations and using dynamic residual modularization for scalable VLA post-training. An important direction of future work is extending DyGRO-VLA to mobile manipulation tasks that involve locomotion tasks before manipulating objects.

Impact Statement

This work aims to advance the field of machine learning and robotics by improving the efficiency and generalization of VLA models through Dynamic Grouped Residual Optimization (DyGRO). The proposed method could accelerate progress toward generalist robotic systems capable of performing a wider range of real-world tasks with greater adaptability and data efficiency.

Potential positive societal impacts include improved accessibility to automation, safer human-robot collaboration, and reduced data collection costs for robotic training. However, as with all advances in general-purpose AI and robotics, there are ethical considerations related to the potential misuse of autonomous systems, labor displacement in automation-sensitive industries, and biases originating from training data. We encourage future research to address these issues through transparent data practices, fairness-aware policy learning, and human-centered deployment standards.

Acknowledgments

This work is supported in part by Shenzhen Science and Technology Program under grant KJZD20240903104008012, Shenzhen Science and Technology Program under grant ZDCY20250901113000001, CUHK-CUHK(SZ)-GDSTC Joint Collaboration Fund No. 2025A0505000053, and Guangdong Key Laboratory of Big Data Computing (2021B1212040002).

References

- Ball, P. J., Smith, L., Kostrikov, I., and Levine, S. Efficient online reinforcement learning with offline data. In *International Conference on Machine Learning*, pp. 1577–1594. PMLR, 2023.
- Belghazi, M. I., Baratin, A., Rajeshwar, S., Ozair, S., Bengio, Y., Courville, A., and Hjelm, D. Mutual information neural estimation. In *International conference on machine learning*, pp. 531–540. PMLR, 2018.
- Bharadhwaj, H., Vakil, J., Sharma, M., Gupta, A., Tulsiani, S., and Kumar, V. Roboagent: Towards sample efficient robot manipulation with semantic augmentations and action chunking. *arxiv*, 2023.
- Black, K., Brown, N., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., Groom, L., Hausman, K., Ichter, B., et al. pi0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- Bohn, C., Freeman, I., Tercan, H., and Meisen, T. Task weighting through gradient projection for multitask learning. In *International Conference on Neural Information Processing*, pp. 317–331. Springer, 2024.
- Chen, T., Chen, Z., Chen, B., Cai, Z., Liu, Y., Li, Z., Liang, Q., Lin, X., Ge, Y., Gu, Z., et al. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025a.
- Chen, Y., Tian, S., Liu, S., Zhou, Y., Li, H., and Zhao, D. Conrft: A reinforced fine-tuning method for vla models via consistency policy. *arXiv preprint arXiv:2502.05450*, 2025b.
- Chen, Z., Ngiam, J., Huang, Y., Luong, T., Kretzschmar, H., Chai, Y., and Anguelov, D. Just pick a sign: Optimizing deep multitask models with gradient sign dropout. *Advances in Neural Information Processing Systems*, 33: 2039–2050, 2020.
- Chen, Z., Niu, R., Kong, H., Wang, Q., Xing, Q., and Fan, Z. Tgrpo: Fine-tuning vision-language-action model via trajectory-wise group relative policy optimization. *arXiv preprint arXiv:2506.08440*, 2025c.
- Chi, C., Xu, Z., Feng, S., Cousineau, E., Du, Y., Burchfiel, B., Tedrake, R., and Song, S. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, pp. 02783649241273668, 2023.
- Florensa, C., Held, D., Wulfmeier, M., Zhang, M., and Abbeel, P. Reverse curriculum generation for reinforcement learning. In *Conference on robot learning*, pp. 482–495. PMLR, 2017.
- Florensa, C., Held, D., Geng, X., and Abbeel, P. Automatic goal generation for reinforcement learning agents. In *International conference on machine learning*, pp. 1515–1528. PMLR, 2018.
- Fu, Y., Zhang, Z., Zhang, Y., Wang, Z., Huang, Z., and Luo, Y. Mergevla: Cross-skill model merging toward a generalist vision-language-action agent. *arXiv preprint arXiv:2511.18810*, 2025.

- Fujimoto, S. and Gu, S. S. A minimalist approach to offline reinforcement learning. *Advances in neural information processing systems*, 34:20132–20145, 2021.
- Guo, Y., Zhang, J., Chen, X., Ji, X., Wang, Y.-J., Hu, Y., and Chen, J. Improving vision-language-action model with online reinforcement learning. *arXiv preprint arXiv:2501.16664*, 2025.
- Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pp. 1861–1870. Pmlr, 2018.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022.
- Huang, D., Fang, Z., Zhang, T., Li, Y., Zhao, L., and Xia, C. Co-rft: Efficient fine-tuning of vision-language-action models through chunked offline reinforcement learning. *arXiv preprint arXiv:2508.02219*, 2025.
- Huang, S., Zhang, Z., Liang, T., Xu, Y., Kou, Z., Lu, C., Xu, G., Xue, Z., and Xu, H. Mentor: Mixture-of-experts network with task-oriented perturbation for visual reinforcement learning. *arXiv preprint arXiv:2410.14972*, 2024.
- Intelligence, P., Black, K., Brown, N., Darpinian, J., Dhabalia, K., et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- Jacobs, R. A., Jordan, M. I., Nowlan, S. J., and Hinton, G. E. Adaptive mixtures of local experts. *Neural computation*, 3(1):79–87, 1991.
- Johannink, T., Bahl, S., Nair, A., Luo, J., Kumar, A., Loskyll, M., Ojea, J. A., Solowjow, E., and Levine, S. Residual reinforcement learning for robot control. In *2019 international conference on robotics and automation (ICRA)*, pp. 6023–6029. IEEE, 2019.
- Kawaharazuka, K., Oh, J., Yamada, J., Posner, I., and Zhu, Y. Vision-language-action models for robotics: A review towards real-world applications. *IEEE Access*, 2025.
- Kim, M. J., Pertsch, K., Karamcheti, S., Xiao, T., Balakrishna, A., Nair, S., Rafailov, R., Foster, E., Lam, G., Sanke, P., et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- Kim, M. J., Finn, C., and Liang, P. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.
- Kong, Y., Ma, G., Zhao, Q., Wang, H., Shen, L., Wang, X., and Tao, D. Mastering massive multi-task reinforcement learning via mixture-of-expert decision transformer. *arXiv preprint arXiv:2505.24378*, 2025.
- Kroemer, O., Niekum, S., and Konidaris, G. A review of robot learning for manipulation: Challenges, representations, and algorithms. *Journal of machine learning research*, 22(30):1–82, 2021.
- Li, H., Zuo, Y., Yu, J., Zhang, Y., Yang, Z., Zhang, K., Zhu, X., Zhang, Y., Chen, T., Cui, G., et al. Simplevla-rl: Scaling vla training via reinforcement learning. *arXiv preprint arXiv:2509.09674*, 2025a.
- Li, Q., Zhou, Z., and Levine, S. Reinforcement learning with action chunking. *arXiv preprint arXiv:2507.07969*, 2025b.
- Liu, B., Liu, X., Jin, X., Stone, P., and Liu, Q. Conflict-averse gradient descent for multi-task learning. *Advances in Neural Information Processing Systems*, 34:18878–18890, 2021.
- Liu, B., Zhu, Y., Gao, C., Feng, Y., Liu, Q., Zhu, Y., and Stone, P. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.
- Liu, J., Gao, F., Wei, B., Chen, X., Liao, Q., Wu, Y., Yu, C., and Wang, Y. What can rl bring to vla generalization? an empirical study. *arXiv preprint arXiv:2505.19789*, 2025.
- Lu, G., Guo, W., Zhang, C., Zhou, Y., Jiang, H., Gao, Z., Tang, Y., and Wang, Z. Vla-rl: Towards masterful and general robotic manipulation with scalable reinforcement learning. *arXiv preprint arXiv:2505.18719*, 2025.
- Luo, J., Hu, Z., Xu, C., Tan, Y. L., Berg, J., Sharma, A., Schaal, S., Finn, C., Gupta, A., and Levine, S. Serl: A software suite for sample-efficient robotic reinforcement learning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 16961–16969. IEEE, 2024.
- Luo, J., Xu, C., Wu, J., and Levine, S. Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning. *Science Robotics*, 10(105):eads5033, 2025.
- Ma, Y., Song, Z., Zhuang, Y., Hao, J., and King, I. A survey on vision-language-action models for embodied ai. *arXiv preprint arXiv:2405.14093*, 2024.
- Nair, A., Gupta, A., Dalal, M., and Levine, S. Awac: Accelerating online reinforcement learning with offline datasets. *arXiv preprint arXiv:2006.09359*, 2020.

- Nakamoto, M., Zhai, S., Singh, A., Sobol Mark, M., Ma, Y., Finn, C., Kumar, A., and Levine, S. Cal-ql: Calibrated offline rl pre-training for efficient online fine-tuning. *Advances in Neural Information Processing Systems*, 36: 62244–62269, 2023.
- Oord, A. v. d., Li, Y., and Vinyals, O. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018.
- Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A., et al. Dinov2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2024.
- Parisotto, E., Ba, J. L., and Salakhutdinov, R. Actor-mimic: Deep multitask and transfer reinforcement learning. *arXiv preprint arXiv:1511.06342*, 2015.
- Peng, X., Chang, C., Wang, F.-Y., and Li, L. Robust multitask learning with sample gradient similarity. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 54(1):497–506, 2023.
- Pertsch, K., Stachowicz, K., Ichter, B., Driess, D., Nair, S., Vuong, Q., Mees, O., Finn, C., and Levine, S. Fast: Efficient action tokenization for vision-language-action models. In *Robotics: Science and Systems (RSS)*, 2025.
- Qu, D., Song, H., Chen, Q., Yao, Y., Ye, X., Ding, Y., Wang, Z., Gu, J., Zhao, B., Wang, D., and Li, X. Spatialvla: Exploring spatial representations for visual-language-action model. In *Robotics: Science and Systems (RSS)*, 2025.
- Sener, O. and Koltun, V. Multi-task learning as multi-objective optimization. *Advances in neural information processing systems*, 31, 2018.
- Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- Shi, L. X., Ichter, B., Equi, M., Ke, L., Pertsch, K., Vuong, Q., Tanner, J., Walling, A., Wang, H., Fusai, N., et al. Hi robot: Open-ended instruction following with hierarchical vision-language-action models. *arXiv preprint arXiv:2502.19417*, 2025.
- Tan, S., Dou, K., Zhao, Y., and Krähenbühl, P. Interactive post-training for vision-language-action models. *arXiv preprint arXiv:2505.17016*, 2025.
- Team, O. M., Ghosh, D., Walke, H., Pertsch, K., Black, K., Mees, O., Dasari, S., Hejna, J., Kreiman, T., Xu, C., et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.
- Team, Q. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2, 2024.
- Tishby, N., Pereira, F. C., and Bialek, W. The information bottleneck method. *arXiv preprint physics/0004057*, 1999.
- Wang, Y., Ding, P., Li, L., Cui, C., Ge, Z., Tong, X., Song, W., Zhao, H., Zhao, W., Hou, P., et al. Vla-adapter: An effective paradigm for tiny-scale vision-language-action model. *arXiv preprint arXiv:2509.09372*, 2025.
- Wang, Z., Tsvetkov, Y., Firat, O., and Cao, Y. Gradient vaccine: Investigating and improving multi-task optimization in massively multilingual models. *arXiv preprint arXiv:2010.05874*, 2020.
- Wu, W., Liu, F., Li, H., Hu, Z., Dong, D., Chen, C., and Wang, Z. Mixture-of-experts meets in-context reinforcement learning. *arXiv preprint arXiv:2506.05426*, 2025.
- Xiao, J., Yang, Y., Chang, X., Chen, R., Xiong, F., Xu, M., Zheng, W.-S., and Zhang, Q. World-env: Leveraging world model as a virtual environment for vla post-training. *arXiv preprint arXiv:2509.24948*, 2025a.
- Xiao, W., Lin, H., Peng, A., Xue, H., He, T., Xie, Y., Hu, F., Wu, J., Luo, Z., Fan, L., et al. Self-improving vision-language-action models with data generation via residual rl. *arXiv preprint arXiv:2511.00091*, 2025b.
- Xing, Y., Luo, X., Xie, J., Gao, L., Shen, H. T., and Song, J. Shortcut learning in generalist robot policies: The role of dataset diversity and fragmentation. In *Conference on Robot Learning, CoRL*, pp. 3239–3266, 2025.
- Yang, L., Shen, D., Cai, C., Yang, F., Gao, T., Zhang, D., and Li, X. Solving token gradient conflict in mixture-of-experts for large vision-language model. *arXiv preprint arXiv:2406.19905*, 2024.
- Yang, R., Xu, H., Wu, Y., and Wang, X. Multi-task reinforcement learning with soft modularization. *Advances in Neural Information Processing Systems*, 33:4767–4777, 2020.
- Yu, T., Kumar, S., Gupta, A., Levine, S., Hausman, K., and Finn, C. Gradient surgery for multi-task learning. *Advances in neural information processing systems*, 33: 5824–5836, 2020.
- Yuan, X., Mu, T., Tao, S., Fang, Y., Zhang, M., and Su, H. Policy decorator: Model-agnostic online refinement for large policy model. *arXiv preprint arXiv:2412.13630*, 2024.
- Zang, H., Wei, M., Xu, S., Wu, Y., Guo, Z., Wang, Y., Lin, H., Shi, L., Xie, Y., Xu, Z., et al. Rlinf-vla: A unified and

efficient framework for vla+ rl training. *arXiv preprint arXiv:2510.06710*, 2025.

Zhai, X., Mustafa, B., Kolesnikov, A., and Beyer, L. Sigmoid loss for language image pre-training. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 11975–11986, 2023.

Zhang, Z., Zheng, K., Chen, Z., Jang, J., Li, Y., Han, S., Wang, C., Ding, M., Fox, D., and Yao, H. Grape: Generalizing robot policy via preference alignment. *arXiv preprint arXiv:2411.19309*, 2024.

Zhao, T. Z., Kumar, V., Levine, S., and Finn, C. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.

A. DyGRO-VLA

A.1. Fusion Module

Following prior work (Wang et al., 2025; Fu et al., 2025), each Fusion layer performs multi-head attention with the action query q_t^a and keys/values from three sources: (1) q_t^a itself, (2) the concatenation of the action hidden state h_t^{act} and proprioceptive tokens $h_t^{\text{pro}} = \text{Enc}(p_t)$, and (3) the conditioning representation h_t^{cond} . A learnable scalar gate dynamically reweights the contribution of the conditioning branch, allowing the model to adaptively integrate task-relevant contextual information.

A.2. Residual RL

Residual MoE. We parameterize the action distribution with an std model that predicts the standard deviation σ_θ , while the Residual MoE provides the mean μ_θ . This formulation defines a factorized Gaussian policy $\pi_\theta(\mathbf{a} \mid s) = \mathcal{N}(\mu_\theta(s), \text{diag}(\sigma_\theta^2(s)))$, which enables stochastic actions during RL exploration and optimization.

Residual Action Space. When refining the base policy with a residual policy, we aim to keep the trajectory close to the original to avoid failure, which means the residual policy should make subtle adjustments. To control this, we bound the output within a specific range. We use the tanh function and mapping the residual action with a hyperparameter α , ensuring it stays within $(-\alpha, \alpha)$.

Offline-to-Online Critic Objective. Our critic for residual RL is a lightweight ResMLP with attention pooling. Following Cal-QL (Nakamoto et al., 2023), we maintain an ensemble of K critics $\{Q_{\theta_i}\}_{i=1}^K$ and optimize them with an h -step Bellman regression term together with a Cal-QL calibration regularizer:

$$\mathcal{L}_Q(\{\theta_i\}) = \frac{1}{K} \sum_{i=1}^K \left(\mathcal{L}_{\text{TD}}(\theta_i) + \lambda_{\text{Cal}} \mathcal{L}_{\text{CalReg}}(\theta_i) \right). \quad (9)$$

Target ensemble and conservative aggregation. For stable target estimation, we maintain a slowly-updated target ensemble $\{\bar{Q}_{\bar{\theta}_j}\}_{j=1}^K$ and define the aggregated target critic as

$$\bar{Q}_{\min}(s, \mathbf{a}) = \min_{j \in \{1, \dots, K\}} \bar{Q}_{\bar{\theta}_j}(s, \mathbf{a}), \quad (10)$$

which yields the backup operator $\mathcal{B}^\pi \bar{Q}_{\min}$.

h -step (chunk) Bellman regression. Each critic is trained by regressing to the shared target:

$$\mathcal{L}_{\text{TD}}(\theta_i) = \mathbb{E}_{(s_t, \mathbf{a}_t, r_t^{(h)}, s_{t+h}) \sim \mathcal{D}} \left[\left(Q_{\theta_i}(s_t, \mathbf{a}_t) - \mathcal{B}^\pi \bar{Q}_{\min}(s_t, \mathbf{a}_t) \right)^2 \right]. \quad (11)$$

The corresponding h -step Bellman backup is

$$\mathcal{B}^\pi \bar{Q}_{\min}(s_t, \mathbf{a}_t) = r_t^{(h)} + \gamma^h \mathbb{E}_{\mathbf{a}' \sim \pi(\cdot | s_{t+h})} [\bar{Q}_{\min}(s_{t+h}, \mathbf{a}')], \quad (12)$$

where the h -step return is

$$r_t^{(h)} = \sum_{i=0}^{h-1} \gamma^i r_{t+i}. \quad (13)$$

Cal-QL calibration regularizer. To enable a smooth transition from offline data to online rollouts, we apply the Cal-QL calibration regularizer to each ensemble member, encouraging high value on policy actions while remaining anchored to dataset actions:

$$\begin{aligned} \mathcal{L}_{\text{CalReg}}(\theta_i) = \mathbb{E}_{s_t \sim \mathcal{D}} \left[\mathbb{E}_{\mathbf{a} \sim \pi(\cdot | s_t)} \left[\max(Q_{\theta_i}(s_t, \mathbf{a}), V^\mu(s_t)) \right] \right. \\ \left. - \mathbb{E}_{\mathbf{a} \sim \mathcal{D}(\cdot | s_t)} [Q_{\theta_i}(s_t, \mathbf{a})] \right]. \end{aligned} \quad (14)$$

Here $\mathbf{a}_t = a_{t:t+h-1}$ denotes an action chunk and $\pi(\mathbf{a} \mid s)$ is the chunk-level policy. We approximate $V^\mu(s_t)$ using Monte Carlo returns from the offline dataset.

B. Proof

Proposition B.1 (Variational Information Bottleneck for Action Regression). *Let Z be a latent representation produced by an encoder $p_\theta(z|o)$, and let $\pi_\theta(a|z)$ denote a probabilistic action decoder. Then, maximizing the IB objective (2) is equivalent to:*

$$\min_{\theta} \mathbb{E}_{p_\theta(z|o)} [-\log \pi_\theta(a|z)] + \lambda_{\text{IB}} \left[\mathbb{E}_{P_{OZ}} [T_\psi(o, z)] - \log \mathbb{E}_{P_O P_Z} [e^{T_\psi(o, z)}] \right]$$

where 1) $T_\psi : \mathcal{O} \times \mathcal{Z} \rightarrow \mathbb{R}$ denotes a neural critic, and 2) P_{OZ} and $P_O P_Z$ denote the joint distribution of O and Z , as well as the product of their marginals.

Proof. Starting from the Information Bottleneck functional:

$$\begin{aligned} & \max_{p_\theta(z|o)} I(Z; A) - \lambda_{\text{IB}} I(Z; O) \\ & \stackrel{(1)}{=} \min_{p_\theta(z|o)} [-I(Z; A) + \lambda_{\text{IB}} I(Z; O)] \\ & \stackrel{(2)}{=} \min_{p_\theta(z|o)} \mathbb{E}_{p(o, a) p_\theta(z|o)} [-\log p(a|z)] + \lambda_{\text{IB}} D_{\text{KL}}(P_{OZ} \| P_O P_Z) \\ & \stackrel{(3)}{=} \min_{p_\theta(z|o)} \mathbb{E}_{(o, a) \sim \mathcal{D}, p_\theta(z|o)} [-\log \pi_\theta(a|z)] + \lambda_{\text{IB}} \sup_{T_\psi} \left\{ \mathbb{E}_{P_{OZ}} [T_\psi(o, z)] - \log \mathbb{E}_{P_O P_Z} [e^{T_\psi(o, z)}] \right\} \\ & \stackrel{(4)}{=} \min_{p_\theta(z|o)} \mathbb{E}_{(o, a) \sim \mathcal{D}, p_\theta(z|o)} [-\log \pi_\theta(a|z)] + \lambda_{\text{IB}} \left(\mathbb{E}_{P_{OZ}} [T_\psi(o, z)] - \log \mathbb{E}_{P_O P_Z} [e^{T_\psi(o, z)}] \right) + \mathbb{C} \\ & \stackrel{(5)}{=} \min_{p_\theta(z|o)} \mathbb{E}_{(o, a) \sim \mathcal{D}, p_\theta(z|o)} [-\log \pi_\theta(a|z)] + \lambda_{\text{IB}} I_{\text{DV}}(O; Z) + \mathbb{C} \end{aligned}$$

where $I_{\text{DV}}(O; Z) = \mathbb{E}_{P_{OZ}} [T_\psi(o, z)] - \log \mathbb{E}_{P_O P_Z} [e^{T_\psi(o, z)}]$ approximates $I(O; Z)$.

- (1) holds because maximizing $I(Z; A) - \beta I(Z; O)$ is equivalent to minimizing its negative.
- (2) holds by substituting the definitions $I(Z; A) = \mathbb{E}[\log p(a|z)] - \mathbb{E}[\log p(a)]$ and $I(Z; O) = D_{\text{KL}}(P_{OZ} \| P_O P_Z)$, where the constant $\mathbb{E}[\log p(a)]$ is omitted since it is independent of θ .
- (3) holds by applying the Donsker–Varadhan (DV) representation of the KL divergence:

$$D_{\text{KL}}(P_{OZ} \| P_O P_Z) = \sup_{T_\psi} \left\{ \mathbb{E}_{P_{OZ}} [T_\psi(o, z)] - \log \mathbb{E}_{P_O P_Z} [e^{T_\psi(o, z)}] \right\}.$$

This introduces the variational critic function $T_\psi(o, z)$.

- (4) holds by absorbing the supremum into optimization over ψ for notation consistency. The additive constant $\mathbb{C}$ encompasses all parameter-independent terms.
- (5) holds by defining $I_{\text{DV}}(O; Z)$ as the Donsker–Varadhan estimate of mutual information. Thus, the resulting loss corresponds to the trainable Information Bottleneck formulation:

$$\mathcal{L}_{\text{base}} = \mathbb{E}[-\log \pi_\theta(a|z)] + \lambda_{\text{IB}} \left(\mathbb{E}_{P_{OZ}} [T_\psi(o, z)] - \log \mathbb{E}_{P_O P_Z} [e^{T_\psi(o, z)}] \right).$$

□



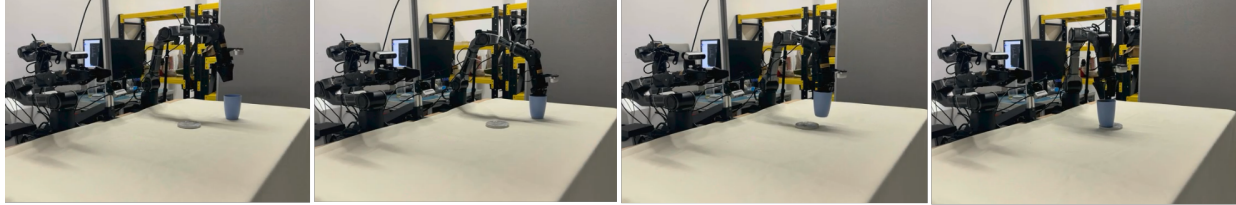
(a) Beat Block Hammer



(b) Pick Dual Bottles



(c) Stack Bowls Two



(d) Place Empty Cup

Figure 8. **Sim-to-real qualitative demonstrations of DyGRO-VLA on RoboTwin2.0.** The policy is trained in simulation and directly deployed in the real world. We show four real-world tasks: Beat Block Hammer, Pick Dual Bottles, Stack Bowls Two, and Place Empty Cup.

C. Real-World Details

Real-World Setups. We deploy the training checkpoint zero-shot on the real robot without any real-world fine-tuning. We evaluate DyGRO-VLA on four real-world tasks from RoboTwin2.0: Beat Block Hammer, Pick Dual Bottles, Stack Bowls Two, and Place Empty Cup. These tasks cover diverse manipulation skills, including contact-rich tool-use, multi-object grasping, object stacking, and goal-conditioned placement. Beat Block Hammer requires accurate tool-object interaction, Pick Dual Bottles evaluates the ability to grasp multiple objects, Stack Bowls Two tests spatial alignment and stable placement, and Place Empty Cup examines pick-and-place generalization in real-world scenes.

Real-World Visualization. As shown in Figure 8, our proposed multi-task training framework achieves effective zero-shot sim-to-real transfer. After being trained only in simulation, DyGRO-VLA can be directly deployed on the real robot without additional real-world data or fine-tuning. The qualitative results demonstrate that the learned multi-task policy generalizes to diverse real-world manipulation scenarios, including tool-use, dual-object picking, stacking, and object placement.

D. Pseudocode

The pseudocode of DyGRO-VLA is provided in Algorithm 1.

Algorithm 1 DyGRO-VLA: IB Pretraining + Online MoRR

Require: $\mathcal{D}_{\text{demo}}, \mathcal{E}, \mathcal{T}; h, N_{\text{on}}, U; \eta_{\text{off}}, \eta_{\pi}, \eta_Q; \lambda_{\text{IB}}, \lambda_{\text{CL}}, \lambda_{\text{LB}}; \theta, \nu, \phi, \psi$
Ensure: Executed chunk $a_t = a_t^{\text{base}} + \Delta a_t$

```

1: Initialize online replay buffer  $\mathcal{D}_{\text{on}} \leftarrow \emptyset$ 
2: Stage I: Offline pretraining
3: for  $i = 1$  to  $N_{\text{off}}$  do
4:   Sample  $(o, l, p, a^*) \sim \mathcal{D}_{\text{demo}}$ 
5:   Sample latent  $z \sim p_{\theta}(z \mid o, l, p)$ 
6:   Predict base chunk  $a^{\text{base}} \leftarrow \pi_{\theta}^{\text{base}}(\cdot \mid z)$ 
7:    $\mathcal{L}_{\text{IB}} \leftarrow \text{DV-IB}(o, z; \nu)$  // variational IB term
8:    $\mathcal{L}_{\text{base}} \leftarrow \mathcal{L}_{\text{BC}}(a^{\text{base}}, a^*) + \lambda_{\text{IB}} \mathcal{L}_{\text{IB}}$ 
9:    $(\theta, \nu) \leftarrow (\theta, \nu) - \eta_{\text{off}} \nabla_{(\theta, \nu)} \mathcal{L}_{\text{base}}$ 
10: end for
11: Freeze  $\theta$  // preserve task-sharing representation
12: Stage II: Online finetuning
13: for  $k = 1$  to  $N_{\text{on}}$  do
14:   Reset  $\mathcal{E}$  and sample task identity  $\zeta \sim \mathcal{T}$ 
15:   Rollout
16:   while not done do
17:     Observe  $(o_t, l_t, p_t)$ 
18:      $(z_t, \tilde{z}_t^T) \leftarrow \text{Enc}_{\theta}(o_t, l_t, p_t)$  // latent + task embedding
19:      $a_t^{\text{base}} \leftarrow \pi_{\theta}^{\text{base}}(\cdot \mid z_t)$ 
20:      $\omega_t \leftarrow \text{Top-}m\text{Softmax}(\text{Router}_{\phi}(\tilde{z}_t^T))$ 
21:     for each expert  $i$  in  $\text{top-}m(\omega_t)$  do
22:       Sample residual chunk  $\tilde{a}_{t,i} \sim \pi_{i,\phi}^{\Delta}(\cdot \mid z_t, a_t^{\text{base}})$ 
23:     end for
24:      $\Delta a_t \leftarrow \sum_{i \in \text{top-}m(\omega_t)} \omega_{t,i} \tilde{a}_{t,i}$ 
25:     Execute chunk  $a_t = a_t^{\text{base}} + \Delta a_t$  for  $h$  steps, receive  $(r_t^{(h)}, o_{t+h}, \text{done})$ 
26:      $(z_{t+h}, \tilde{z}_{t+h}^T) \leftarrow \text{Enc}_{\theta}(o_{t+h}, l_{t+h}, p_{t+h})$ 
27:     Store  $(z_t, \tilde{z}_t^T, \zeta, \omega_t, \Delta a_t, r_t^{(h)}, z_{t+h}, \text{done})$  into  $\mathcal{D}_{\text{on}}$ 
28:   end while
29:   Updates
30:   for  $u = 1$  to  $U$  do
31:     Sample minibatch  $\mathcal{M} \sim \mathcal{D}_{\text{on}}$ 
32:      $\mathcal{L}_Q \leftarrow \mathcal{L}_{\text{TD/Cal-QL}}(\mathcal{M}; \psi, \theta, \phi)$ 
33:      $\psi \leftarrow \psi - \eta_Q \nabla_{\psi} \mathcal{L}_Q$ 
34:      $\mathcal{L}_{\text{RL}} \leftarrow \mathcal{L}_{\text{actor}}(\mathcal{M}; \psi, \theta, \phi)$  // SAC-style on augmented policy
35:      $\mathcal{L}_{\text{CL}} \leftarrow \mathcal{L}_{\text{InfoNCE}}(\mathcal{M}; \phi)$  // shape  $\tilde{z}^T$ 
36:      $\mathcal{L}_{\text{LB}} \leftarrow \mathcal{L}_{\text{load-balance}}(\mathcal{M}; \phi)$  // prevent expert collapse
37:      $\phi \leftarrow \phi - \eta_{\pi} \nabla_{\phi} (\mathcal{L}_{\text{RL}} + \lambda_{\text{CL}} \mathcal{L}_{\text{CL}} + \lambda_{\text{LB}} \mathcal{L}_{\text{LB}})$ 
38:   end for
39: end for
40: return  $\pi(a_t \mid o_t, l_t, p_t) = \pi_{\theta}^{\text{base}}(\cdot \mid z_t) + \Delta a_t$ 

```
